

Адаптированная стратегия Fumadocs для кастомной документации на Next.js

Executive summary

Для вашей задачи Fumadocs подходит очень хорошо именно потому, что он не навязывает «тему целиком», а разделяет систему на **UI**, **Core** и **Content Source**. Официальная философия Fumadocs — это «framework that you can break»: UI-компоненты можно использовать как готовые блоки, а при необходимости заменять части через CLI, слоты и headless-слой. Для проекта, где документация должна выглядеть кастомно, но при этом не хочется заново писать sidebar, TOC, page tree и MDX-инфраструктуру, это как раз сильная сторона платформы. ¹

Практически это означает такой базовый подход: оставить **Fumadocs MDX** источником контента, собрать `source` через `loader()` в `lib/source.ts`, а затем в App Router использовать `DocsLayout` для общего каркаса документации и `DocsPage` для страницы контента. В этом сценарии левый sidebar приходит из `source.getPageTree()`, центральная область рендерит `page.data.body`, а правый TOC получает структуру заголовков из `page.data.toc`, которую Fumadocs MDX отдаёт из коробки. ²

На уровне Next.js App Router наиболее естественная схема такая: корневой `app/layout.tsx` держит `RootProvider`; сегмент документации, например `app/docs/layout.tsx`, рендерит `DocsLayout` и получает дерево через `source.getPageTree()`; страница `app/docs/[...slug]/page.tsx` спрашивает `source.getPage(slugs)`, а если страница отсутствует — вызывает `notFound()`. Для статической генерации маршрутов `source.generateParams()` специально предназначен для Next.js `generateStaticParams`. Поскольку в App Router страницы и layout по умолчанию серверные, именно серверный сценарий для docs-роутов здесь является основным и наиболее простым. ³

Самый важный практический вывод: **не начинайте с headless-компонентов**. Для вашей задачи разумный путь — сначала собрать документацию на готовых `DocsLayout` и `DocsPage`, затем довести дизайн через `props`, `containerProps`, CSS variables, `id / data-*` селекторы и, только если этого реально не хватит, точно заменить slot-компоненты sidebar/TOC/breadcrumb/footer или перейти на headless TOC/Breadcrumb/PageTree. Сама официальная документация Fumadocs советует для «normal usages» брать Fumadocs UI, а не headless-слой. ⁴

Ниже я исхожу из следующих вводных. Многоязычность — **не указано**. Версионирование docs — **не указано**. Наличие отдельной публичной поисковой страницы `/docs/search` — **не указано**. Интеграция с GitHub edit links — **не указано**. Тип поиска — **не указано**. Поэтому рекомендации ниже ориентированы на базовый сценарий: одна документация внутри существующего Next.js-приложения, MDX как источник истины, серверный рендер, кастомный дизайн поверх Fumadocs UI.

Карта чтения официальной документации

Ниже — страницы официальной документации Fumadocs, которые действительно помогут именно для вашей задачи. Я разделил их по приоритету: **обязательно, очень желательно, по ситуации**.

Страница	Приоритет	Что даёт	На что смотреть именно вам
What is Fumadocs 5	Обязательно	Объясняет модель UI + Core + Content Source и философию composable/customizable docs framework	Подтверждает, что кастомный дизайн поверх готовых блоков — штатный сценарий, а не «борьба с темой»
Fumadocs MDX – Getting Started 6	Обязательно	Объясняет <code>doc</code> , <code>meta</code> , <code>docs</code> , типобезопасный вывод и встроенные поля MDX (<code>frontmatter</code> , <code>toc</code> , <code>structuredData</code>)	Это базовый слой для вашего центрального MDX и правого ТОС
Next.js для Fumadocs MDX 7	Обязательно	Показывает <code>createMDX</code> , интеграцию с Next.js, <code>.source</code> , сборку <code>loader({ baseUrl, source })</code>	Нужен, чтобы проверить, что текущая установка правильно готовит отдельный docs-роут
Server Entry 8	Обязательно	Показывает, как использовать <code>collections/server</code> , <code>docs.toFumadocsSource()</code> , <code>page.data.body</code> и доступ к <code>page.data.*</code>	Это ваш основной режим для App Router; на него и стоит опираться
Page Slugs & Page Tree 9	Обязательно	Объясняет, как из файлов и <code>meta.json</code> строятся slug-и и sidebar tree	Это ключ к левому sidebar, category-страницам, root folders, порядку разделов, separators и internal/external links
Loader API 10	Обязательно	Описывает <code>getPage</code> , <code>getPages</code> , <code>getPageTree</code> , <code>generateParams</code> , <code>getLanguages</code>	Это ваш ежедневный рабочий API между контентом и App Router
Docs Layout 11	Обязательно	Готовый layout документации с sidebar и mobile header	Главное: <code>tree</code> , <code>sidebar</code> , <code>tabs</code> , <code>tabMode</code> , <code>containerProps</code> , <code>nav</code> , <code>links</code> , <code>slots</code> ; особенно важно понять, как далеко можно уехать без полной замены layout

Страница	Приоритет	Что даёт	На что смотреть именно вам
Docs Page 12	Обязательно	Готовая страница документации с TOC, breadcrumb, footer и контейнером статьи	Для вас критичны: <code>toc</code> , <code>tableOfContent</code> , <code>tableOfContentPopover</code> , <code>full</code> , <code>breadcrumb</code> , <code>footer</code> , <code>slots</code>
Root Provider 13	Очень желательно	Даёт контекст Fumadocs UI, тему и search dialog	Нужен в корневом layout; если его недооценить, потом будут странности с темой и поиском
Customize UI 14	Очень желательно	Показывает правильную лестницу кастомизации: props → CSS → CLI customize/slots	Самая практичная страница для кастомного дизайна без ухода в полный reimplementation
Themes 15	Очень желательно	CSS-переменные, темы, ширина layout, цветовая система, типографика	Для кастомной docs-страницы это лучший способ быстро приблизить UI к дизайну без переписывания компонентов
Components 16	Очень желательно	Даёт MDX-компоненты и <code>createRelativeLink</code>	Читайте, если хотите максимально использовать встроенные callout/cards/tabs/codeblocks и корректные относительные ссылки в MDX
Layout Links 17	По ситуации	Настройка верхних ссылок и меню	Нужно, если docs должны жить внутри общей навигации приложения
Navbar 18	По ситуации	Настройка/замена navbar и прозрачного режима	Нужно, если дизайн шапки документации не совпадает с дефолтом
Search и Search UI 19	По ситуации	Как подключается поиск и как кастомизируется диалог	Актуально, если хотите полнотекстовый поиск как часть docs UX
Built-in Search 20	По ситуации	Стандартный search backend на Orama и <code>app/api/search/route.ts</code>	Нужно, если нужен поиск без стороннего SaaS
Headless TOC 21	По ситуации	Низкоуровневый TOC с active-anchor observer и auto-scroll	Читайте только если стандартный TOC по DOM/UX не подходит

Страница	Приоритет	Что даёт	На что смотреть именно вам
Headless Breadcrumb 22	По ситуации	Хук <code>useBreadcrumb(pathname, tree)</code>	Нужен, если вы хотите свой breadcrumb вне <code>DocsPage</code>
Headless Page Tree 23	По ситуации	Формат дерева навигации: <code>root/page/folder/separator</code>	Нужен, если вы сами рисуете sidebar или строите нестандартную категорийную навигацию

Если упростить до совсем короткого маршрута чтения, то для вашей реализации я бы читал в таком порядке: **What is Fumadocs → Fumadocs MDX Getting Started → Next.js → Server Entry → Page Slugs & Page Tree → Loader API → Docs Layout → Docs Page → Root Provider → Customize UI → Themes → Components**. Этого достаточно, чтобы спроектировать сильную первую версию без провала в необязательные разделы. 24

Рекомендуемая архитектура для Next.js App Router

В App Router layout и page по умолчанию являются **Server Components**, а динамические параметры передаются в `layout`, `page` и `generateMetadata`; `generateStaticParams` можно объявлять в `page`, `layout` и route handlers. Для Fumadocs это очень удачное совпадение: `loader()` — серверный API, `source.getPage()` и `source.getPageTree()` естественно вызываются на сервере, а клиентский код нужен только там, где вы добавляете интерактивность поверх уже собранной docs-страницы. 25

Практически рекомендуемая связка выглядит так:

корневой `app/layout.tsx` содержит `RootProvider`;
 сегмент документации `app/docs/layout.tsx` рендерит `DocsLayout` и получает `tree` из `source.getPageTree()`;
 элемент маршрута `app/docs/[...slug]/page.tsx` вызывает `source.getPage(slugs)`, рендерит `page.data.body`, передаёт `page.data.toc` в `DocsPage` и при отсутствии страницы вызывает `notFound()`. Такой сценарий полностью укладывается в официальные рекомендации Fumadocs MDX Server Entry и Loader API. 26

flowchart TD

```

A[app/layout.tsx] --> B[RootProvider]
B --> C[app/docs/layout.tsx]
C --> D[DocsLayout]
D --> E[source.getPageTree]
C --> F[app/docs/[...slug]/page.tsx]
F --> G[source.getPage(slugs)]
G --> H[page.data.body]
G --> I[page.data.toc]
F --> J[DocsPage]
J --> K[Sidebar слева]
J --> L[MDX по центру]
```

```
J --> M[ТОС справа]
G --> |page not found| N[notFound]
```

Диаграмма выше отражает наиболее «нативный» для Fumadocs и Next.js путь: layout отвечает за пространственную рамку и навигацию, page — за конкретный MDX-документ, а source — за связь файловой структуры с роутингом и UI-деревом. ²⁷

API и где их вызывать

API / функция	Что делает	Где вызывать в App Router	Сервер / клиент	Практический смысл задачи
<code>createMDX(...)</code> ²⁸	Подключает Fumadocs MDX к Next config	<code>next.config.mjs</code>	Серверная конфигурация сборки	Основная генерация трогает при на компиляции пути к
<code>defineDocs(...)</code> / <code>defineCollections(...)</code> ²⁹	Описывает content collections	<code>source.config.ts</code>	Сборка / конфигурация	Определяет какие meta-сущности участвуют в документах
<code>loader({ source, baseUrl })</code> ³⁰	Собирает unified source API	<code>lib/source.ts</code>	Сервер	Центральная точка docs с
<code>source.getPage(slugs)</code> ³¹	Возвращает страницу по slug-массиву	<code>app/docs/[...slug]/page.tsx</code>	Сервер	Основная загрузка конкретной документации
<code>source.getPages()</code> ³¹	Даёт список всех страниц	server functions, sitemap/search/category pages	Сервер	Полезно для поиска, lister-с category
<code>source.getPageTree()</code> ¹⁰	Возвращает дерево навигации	<code>app/docs/layout.tsx</code> , иногда custom breadcrumb/sidebar plumbing	Сервер	Источник и соседние навигации
<code>source.generateParams()</code> ³²	Готовит output для <code>generateStaticParams</code>	<code>app/docs/[...slug]/page.tsx</code>	Сервер	Самый последний путь к статическим генерациям маршрутов

API / функция	Что делает	Где вызывать в App Router	Сервер / клиент	Практический смысл задачи
<code>page.data.body</code> ⁸	React Server Component для рендера MDX	<code>app/docs/[[...slug]]/page.tsx</code>	Сервер	Центр MDX-содержимого
<code>page.data.toc</code> ³³	Готовый TOC из MDX	<code>app/docs/[[...slug]]/page.tsx</code>	Сервер → передаётся в UI	Правильно обрабатывает собственные размеры заголовков
<code>page.data.frontmatter</code> / <code>page.data.title</code> / <code>page.data.description</code> ³⁴	Метаданные документа	<code>page.tsx</code> , <code>generateMetadata</code>	Сервер	Заголовок, описание, Graph, breadcrumbs, блоки
<code>generateMetadata(...)</code> ³⁵	Генерирует metadata для page/layout	<code>page.tsx</code> или <code>layout.tsx</code>	Сервер	Логично title/description/docs-content/page
<code>notFound()</code> ³⁶	Прерывает рендер и отдаёт not-found UI	<code>page.tsx</code> docs-сегмента	Сервер	Корректно обрабатывает несущий slug
<code>RootProvider</code> ¹³	Контекст темы и поиска	<code>app/layout.tsx</code>	Серверный layout, клиентский контекст внутри	База данных search/docs-p
<code>createRelativeLink(source, page)</code> ¹⁶	Правильно обрабатывает относительные ссылки в MDX	<code>app/docs/[[...slug]]/page.tsx</code> при настройке MDX components	Только Server Component	Нужен ваш файл build/.otl file.
<code>useBreadcrumb(pathname, tree)</code> ²²	Строит breadcrumb items по page tree	custom breadcrumb component	Клиент	Нужен если и breadcrumb Docs
<code>AnchorProvider</code> / <code>ScrollProvider</code> / <code>TOCItem</code> ²¹	Headless TOC с active-anchor tracking	custom TOC component	Обычно клиент	Нужен придётся самому м TOC, а внеш

Ключевая архитектурная рекомендация здесь простая: для docs-страницы почти всё стоит оставить на сервере. Переход на `collections/browser` или на fully custom client-side content

loading оправдан в других React-фреймворках или при особых ограничениях, но даже документация Browser Entry прямо рекомендует использовать RSC whenever possible для лучшей производительности и меньшей гидратации. ³⁷

source.config.ts и модель данных

Самое полезное здесь — разделять две вещи: **какие данные парсятся** и **где им назначается URL**. В Fumadocs MDX `source.config.ts` отвечает за коллекции и правила обработки контента, а URL-домен документации (`baseUrl`) задаётся уже не там, а в `loader()` внутри `lib/source.ts`. Это частый источник путаницы, и для вашей задачи важно зафиксировать его сразу. ³⁸

Ключевые поля и что они дают

Поле / сущность	Где задаётся	Что означает	Что даёт в рантайме	Применение в вашей задаче
<code>defineDocs({ dir, docs, meta })</code> ³⁹	<code>source.config.ts</code>	Объединённая docs-коллекция Fumadocs	Порождает <code>docs</code> + <code>meta</code> выходы, пригодные для <code>toFumadocsSource()</code>	Лучший выбор для отдельной документации в одном приложении
<code>dir</code> ⁴⁰	<code>source.config.ts</code>	Корневая папка контента	Определяет, откуда читаются MDX и meta-файлы	Например, весь docs-контент может жить в <code>content/docs</code>
<code>docs</code> внутри <code>defineDocs</code> ⁴¹	<code>source.config.ts</code>	Параметры <code>doc</code> -коллекции	Управляет MDX/Markdown файлами, frontmatter schema, postprocess, mdxOptions	Источник центрального MDX, ТОС и page-level metadata
<code>meta</code> внутри <code>defineDocs</code> ⁴²	<code>source.config.ts</code> и <code>meta.json</code>	Параметры <code>meta</code> -коллекции и файлов папок	Управляет отображением папок, порядком страниц, tabs, separators	Источник левой навигации и category-структуры
<code>schema</code> для <code>docs</code> / <code>meta</code> ⁴¹	<code>source.config.ts</code>	Валидация и расширение схемы данных	Делает frontmatter/meta типобезопасными	Полезно, если в дизайне нужны дополнительные поля вроде <code>badge</code> , <code>index</code> , <code>hero</code> , <code>toc</code> , <code>false</code>

Поле / сущность	Где задаётся	Что означает	Что даёт в рантайме	Применение в вашей задаче
Built-in exports: <code>frontmatter</code> , <code>toc</code> , <code>structuredData</code> , <code>extractedReferences</code> 6	Генерируются из MDX	Стандартные данные страницы	<code>frontmatter</code> для заголовков/SEO, <code>toc</code> для TOC, <code>structuredData</code> для search	Минимизировать свой парсинг свой контентный pipeline
<code>title</code> , <code>description</code> , <code>icon</code> во <code>frontmatter</code> 43	В <code>.mdx</code> файле	Свойства страницы для <code>page tree</code>	Попадают в <code>page.data</code> и используются при построении дерева	Это минимальный набор, который стоит держать почти у каждой документации страницы
Slug из пути файла 44	Автоматически	<code>./dir/page.mdx</code> <code>-></code> <code>['dir', 'page']</code> , <code>./dir/index.mdx</code> <code>-></code> <code>['dir']</code>	Даёт <code>route params</code> и URL-структуру	Основной аргумент в пользу <code>[[...slug]]</code>
<code>meta.json: title</code> , <code>icon</code> , <code>defaultOpen</code> , <code>collapsible</code> , <code>pages</code> , <code>pagesIndex</code> 45	В каждой папке docs	Настройка папки и порядка <code>items</code>	Влияет на sidebar, кликабельность категории, порядок, <code>separators</code> и <code>links</code>	Главный инструмент управления левым меню без кастомного sidebar
<code>root: true</code> в <code>meta.json</code> 46	В папке-корне	Помечает папку как <code>root folder</code>	В Fumadocs UI такие папки рендерятся как <code>layout tabs</code>	Использовать только если вас действительно несколько крупных разделов docs
<code>baseUrl</code> 30	Не в <code>source.config.ts</code> , а в <code>lib/source.ts</code> через <code>loader()</code>	Базовый URL документации	Формирует <code>page.url</code> и путь в <code>page tree</code>	Для отдельного docs-раздела, обычно это <code>docs</code>

Какие данные вы реально получите из страницы

Данные	Откуда берутся	Какую роль играют
<code>page.data.body</code> 8	Компиляция MDX	Центральный контент документа

Данные	Откуда берутся	Какую роль играют
<code>page.data.title</code> / <code>description</code> ⁴⁷	Frontmatter и schema	<code><DocsTitle></code> , <code><DocsDescription></code> , metadata
<code>page.data.toc</code> ³³	Автоматически из заголовков документа	Правый TOC
<code>page.data.structuredData</code> ⁴⁸	Автоматически из контента	Поиск и search indexing
<code>page.url</code> / <code>page.slugs</code> ⁴⁹	<code>loader()</code> + путь файла	Роутинг, активный sidebar item, breadcrumbs

С точки зрения проектирования это означает следующее: **не надо изобретать собственную «контентную модель docs» раньше времени**. Пока у вас нет уникальных требований к полям frontmatter, достаточно опереться на стандартный набор `title`, `description`, `icon`, встроенный `toc` и `structuredData`, а организацию боковой навигации делать через `meta.json`. Во многих командах именно этот шаг убирает 70–80% лишней работы. ⁵⁰

DocsLayout и DocsPage как основа кастомного дизайна

`DocsLayout` — это каркас документации: sidebar плюс mobile-only navbar/header. Его базовый режим очень хорошо совпадает с вашей целью «слева навигация, в центре контент, справа TOC», потому что сам контентную страницу он не навязывает — он лишь задаёт окружение и ожидает `tree`. Кроме того, документация явно показывает, что `DocsLayout` можно кормить `source.getPageTree()`, а sidebar items рендерятся именно из page tree. ⁵¹

`DocsPage` — это уже базовый элемент конкретной docs-страницы. Он включает **TOC**, **footer** и **breadcrumb**; может принимать готовый `toc`, умеет переключаться в `full`-режим, а TOC, footer, breadcrumb и container можно заменять через `slots`. Для вашей задачи это почти идеальная отправная точка: берёте готовую механику страницы и меняете только то, что реально влияет на бренд-дизайн. ¹²

Что важно в DocsLayout

Параметр / зона	Зачем нужен вам	Когда трогать
<code>tree</code> ⁵²	Источник sidebar navigation	Всегда; почти всегда это <code>source.getPageTree()</code>
<code>sidebar.enabled</code> ⁵³	Включение/отключение сайдбара	Если docs-landing и article pages должны отличаться
<code>sidebar.components</code> ⁵⁴	Частичная замена элементов sidebar	Когда дизайн sidebar нестандартный, но tree-логика устраивает
<code>sidebar.banner</code> ⁵⁵	Зона баннера в сайдбаре	Для СТА, версии, статуса beta, внутренних ссылок

Параметр / зона	Зачем нужен вам	Когда трогать
<code>tabs</code> и <code>tabMode</code> ⁵⁶	Верхнее переключение крупных root-разделов	Трогать только если docs реально делятся на несколько крупных «документаций»
<code>containerProps</code> ⁵⁷	Класс/атрибуты контейнера layout	Первый безопасный уровень кастомного дизайна
<code>nav</code> , <code>links</code> , <code>githubUrl</code> ⁵⁸	Верхняя навигация docs и интеграция с вашим приложением	Если docs не полностью изолированы и должны жить в общей IA сайта
<code>slots</code> ⁵⁹	Более глубокая замена частей layout	Когда props и CSS уже не хватает

Что важно в `DocsPage`

Параметр / зона	Зачем нужен вам	Когда трогать
<code>toc</code> ⁶⁰	Передача массива заголовков	Всегда, если нужен правый ТОС
<code>tableOfContent</code> / <code>tableOfContentPopover</code> ⁶⁰	Настройка внешнего вида/поведения ТОС	Когда нужен другой стиль ТОС без полного переписывания
<code>full</code> ⁶¹	Убирает фиксированный режим и переводит ТОС в popover	Если нужны более широкие статьи или нестандартная grid-композиция
<code>footer</code> ⁶²	Настройка next/prev navigation	Если хотите оставить механику соседних страниц, но сменить сору/вид
<code>breadcrumb</code> ⁶³	Настройка показа breadcrumb	Если breadcrumb нужен не всегда или должен быть проще
<code>slots.toc</code> ⁶⁴	Замена ТОС-компонентов	Когда default ТОС слишком жёстко ограничивает дизайн
<code>slots.footer</code> ⁶⁵	Замена footer	Когда нужен собственный prev/next layout
<code>slots.breadcrumb</code> ⁶⁶	Замена breadcrumb	Когда breadcrumb должен жить в другом месте или иметь другую DOM-структуру
<code>slots.container</code> ⁶⁷	Замена article wrapper	Когда нужен свой контейнер статьи, собственные отступы, sticky-зоны и пр.

Что важно в `RootProvider`

`RootProvider` должен жить в корневом layout. Он отвечает за контекст компонентов Fumadocs UI, в том числе за `next-themes` и framework provider; через него также настраивается или отключается search dialog и theme provider. На практике это означает, что даже если docs-секция у вас отдельная, `RootProvider` лучше держать выше неё, а не «внутри /docs», чтобы не плодить разный UI-контекст в приложении. ¹³

Когда реально переходить на headless

Переход на headless-компоненты оправдан не тогда, когда «хочется чистоты», а когда штатные layout/page блоки **мешают** целевому UX. Для вашей задачи я бы использовал такой чеклист.

Причина	Что брать	Почему
Нужен полностью свой DOM и ритм правого TOC, а не просто другой стиль	Headless TOC ²¹	Он даёт active-anchor observer, auto-scroll и <code>TOCItem</code> , но не навязывает финальную вёрстку
Breadcrumb должен жить не там, где его размещает <code>DocsPage</code> , или зависеть от другого client state	Headless Breadcrumb ²²	<code>useBreadcrumb(pathname, tree)</code> даёт только данные, а рендерите вы уже как угодно
Левое меню должно быть кардинально иным: карточки, много колонок, mega-sidebar, category hubs	Headless Page Tree ⁶⁸	Page tree остаётся источником истины, но UI sidebar собирается самостоятельно
Нужно хранить в sidebar/TOC сложные локальные интеракции и stateful поведение, которое не вписывается в slots	Headless + custom client components ⁶⁹	Тогда комфортнее взять данные из Core и построить свою интерактивную оболочку
Дизайн требует таких CSS-вмешательств, которые опираются на хрупкую DOM-структуру дефолтных компонентов	Выборочно заменить slot через CLI ⁷⁰	Это безопаснее, чем ломать тему глубокими селекторами

Если ни один из этих пунктов не выполняется, то headless, скорее всего, будет преждевременной оптимизацией. Официальные материалы Fumadocs UI и Customize UI прямо подталкивают сначала использовать props, CSS и slots, а не уходить в полную переработку. ¹⁴

Типовые роуты, логика загрузки данных и путь внедрения

Для вашего кейса я рекомендую строить docs-раздел вокруг **одного catch-all route** внутри сегмента `/docs`. Официальный Loader API прямо показывает использование `app/[...slug]/page.tsx` с `source.generateParams()`, а правила slug generation в Fumadocs естественно сочетаются с optional catch-all динамическим сегментом Next.js. Это убирает дублирование логики между `index` и вложенными docs-страницами. ⁷¹

flowchart LR

```
A[/docs/] --> B[index.mdx]
A --> C[/docs/guide]
C --> D[guide/index.mdx или pagesIndex]
C --> E[/docs/guide/installation]
C --> F[/docs/guide/configuration]
A --> G[/docs/components]
G --> H[components/index.mdx]
A --> I[/api/search]
X[Неизвестный slug] --> Y[notFound]
```

Рекомендуемые роуты

Роут / файл	Назначение	Логика загрузки данных	Статус для вашей задачи
<code>app/layout.tsx</code>	Корневой layout приложения	Держит <code>RootProvider</code> ; управляет theme/search глобально ¹³	Обязателен
<code>app/docs/layout.tsx</code>	Общий каркас docs-раздела	Вызывает <code>source.getPageTree()</code> и отдаёт <code>tree</code> в <code>DocsLayout</code> ⁷²	Обязателен
<code>app/docs/[[...slug]]/page.tsx</code>	Одна точка входа для index и вложенных docs-страниц	Берёт slug из params, вызывает <code>source.getPage(slugs)</code> , рендерит body/toc/frontmatter, при <code>null</code> вызывает <code>notFound()</code> ⁷³	Обязателен
<code>generateStaticParams</code> в docs page	Предгенерация docs-маршрутов	Возвращает <code>source.generateParams()</code> ³²	Очень желательно
<code>generateMetadata</code> в docs page	SEO/соц. metadata	Строит title/description из <code>page.data.title</code> / <code>page.data.description</code> ; Next.js позволяет делать это в <code>page/layout</code> ⁷⁴	Очень желательно
<code>app/docs/not-found.tsx</code> или общий <code>app/not-found.tsx</code>	UI для отсутствующих страниц docs	Показывается после <code>notFound()</code> из docs route segment ⁷⁵	Желательно
<code>app/api/search/route.ts</code>	Search backend для docs	Для built-in search используется <code>createFromSource(source)</code> либо <code>createSearchAPI(...)</code> с <code>structuredData</code> ²⁰	По ситуации

Роут / файл	Назначение	Логика загрузки данных	Статус для вашей задачи
<code>/docs/search</code> как page	Отдельная поисковая страница	Не обязательна: Fumadocs по умолчанию делает search dialog через <code>RootProvider</code> / Search UI, а не отдельную страницу ⁷⁶	Не указано
<code>category</code> -страницы вроде <code>/docs/guide</code>	Лэндинг раздела / индекс папки	Обычно реализуются как <code>guide/index.mdx</code> или через <code>pagesIndex</code> в <code>meta.json</code> ; это не особый тип роута, а обычная docs-страница папки ⁷⁷	Рекомендовано, если нужны разделы

Поток рендера docs-страницы

```

flowchart TD
    A[Запрос в /docs/...slug] --> B[Next.js page params]
    B --> C[source.getPage(slugs)]
    C -->|нет страницы| D[notFound()]
    C -->|страница есть| E[page.data.title/description]
    C --> F[page.data.toc]
    C --> G[page.data.body]
    H[source.getPageTree()] --> I[DocsLayout sidebar]
    F --> J[DocsPage TOC]
    E --> K[DocsTitle/DocsDescription + metadata]
    G --> L[MDX render]
    I --> M[Итоговая docs-страница]
    J --> M
    K --> M
    L --> M

```

Лестница кастомизации UI

Официальный гид Customize UI задаёт очень здоровую последовательность: сначала проверить, хватает ли props, затем использовать CSS, а затем обращаться к CLI customize и slot-level overrides. Это важнее, чем кажется, потому что большая часть боли в кастомных docs-системах возникает именно от слишком раннего ухода в полную замену компонентов. ¹⁴

Уровень	Что делать	Плюсы	Риски / trade-offs
<code>props</code> и штатные options ⁷⁸	<code>containerProps</code> , <code>sidebar</code> , <code>tabs</code> , <code>nav</code> , <code>breadcrumb</code> , <code>tableOfContent</code> , <code>full</code>	Самый стабильный и дешёвый путь	Ограничен доступными пропсами

Уровень	Что делать	Плюсы	Риски / trade-offs
CSS variables и селекторы <code>id</code> / <code>data-*</code> ⁷⁹	Менять ширины, цвета, nav height, отдельные визуальные зоны	Быстро даёт сильный визуальный эффект	Если стили слишком завязаны на DOM-структуру, возрастает хрупкость при обновлениях
Slot overrides через CLI ⁸⁰	Заменять TOC/footer/breadcrumb/container/sidebar parts точно	Хороший баланс между контролем и повторным использованием	Нужно сопровождать локальную копию slot-компонента
Полный <code>customize</code> layout через CLI ⁸¹	Забрать layout в проект и менять всё	Максимальный контроль	Вы берёте на себя бремя обновлений и совместимости

Для типичной кастомной документации я бы целился в такой предел:

layout/page на штатных компонентах,
custom design на CSS variables + containerProps,
небольшие локальные замены slots,

и только в самом конце — полный `customize` UI. Это даёт именно тот баланс, который вы описали: «максимально использовать готовые Fumadocs элементы, чтобы не писать свои специфичные компоненты». ⁸²

Пошаговый план внедрения

Шаг	Что сделать	Точка проверки
Проверить текущую установку MDX	Убедиться, что <code>createMDX</code> подключён, <code>.source</code> генерируется, а <code>source.config.ts</code> действительно описывает docs-контент ⁷	MDX из <code>content/docs</code> доступен через <code>collections/server</code>
Зафиксировать контентную структуру	Решить, где будут <code>index.mdx</code> , папки разделов, <code>meta.json</code> , нужны ли root folders/tabs ⁸³	Можно вручную нарисовать итоговый sidebar на бумаге и сопоставить его с файлами
Собрать <code>lib/source.ts</code>	Настроить <code>loader({ source: docs.toFumadocsSource(), baseUrl: '/docs' })</code> ⁸⁴	<code>source.getPageTree()</code> и <code>source.getPage()</code> дают ожидаемые данные
Поднять layout docs-сегмента	Сделать отдельный docs layout на <code>DocsLayout</code> и подключить page tree ⁸⁵	Sidebar появляется и отражает структуру файлов
Поднять docs page route	На <code>[[...slug]]</code> загружать страницу, отдавать <code>DocsPage</code> , <code>body</code> , <code>toc</code> , <code>title</code> , <code>description</code> , <code>notFound</code> ⁸⁶	<code>index.mdx</code> и вложенные страницы открываются через один маршрут

Шаг	Что сделать	Точка проверки
Подключить metadata и pre-render	Использовать <code>generateMetadata</code> и <code>source.generateParams()</code> ⁸⁷	Заголовки вкладки и статические маршруты совпадают с контентом
Провести визуальную кастомизацию	Сначала <code>containerProps</code> и CSS vars, потом аккуратные <code>id/data-*</code> стили, потом slots при необходимости ⁸⁸	Дизайн близок к макету без полной замены layout/page
Добавить поиск и polish	Через <code>RootProvider</code> + Search UI, а backend — через built-in search route при необходимости ⁸⁹	Документация удобна в использовании, а не только красиво выглядит

Что можно пропустить на этом этапе

Для вашей текущей цели можно спокойно **не читать** следующие блоки, если только задачи не появятся позже.

Раздел	Почему можно пропустить сейчас
Browser Entry и Dynamic Entry ⁹⁰	Для App Router серверный сценарий проще и рекомендованнее; browser entry нужен скорее для non-RSC сценариев
Notebook Layout, Flux Layout, Home Layout ⁹¹	Вы уже знаете, что хотите кастомную docs-страницу на основе стандартного docs layout, а не альтернативную opinionated тему
OpenAPI / AsyncAPI / Story ⁹²	Это генераторы контента под специальные сценарии, не нужны для обычной MDX-документации
Internationalization ⁹³	Полезно только если документация будет многоязычной; сейчас это не указано
Export PDF / EPUB / RSS / OG Image / Feedback / AI & LLMs ⁹²	Это этап polish и расширений, а не база для sidebar + MDX + TOC
Глубокие headless-разделы ⁹⁴	Имеет смысл читать только после того, как станет ясно, что <code>DocsLayout</code> и <code>DocsPage</code> вас реально ограничивают

Итоговая рекомендация в одной фразе такая: **постройте первую версию docs как server-first App Router сегмент на `DocsLayout` + `DocsPage` + `loader()` + `meta.json`, дизайн доводите через props/CSS/slots, а headless оставьте как запасной выход, а не стартовую точку**. Именно этот путь лучше всего соответствует и вашим целям, и тому, как сам Fumadocs предлагает использовать свою архитектуру. ⁹⁵

¹ ⁵ ²⁴ ⁸² ⁹⁵ What is Fumadocs | Fumadocs

<https://www.fumadocs.dev/docs/what-is-fumadocs>

² ⁷ ²⁸ ³⁰ ⁸⁴ Next.js | Fumadocs

<https://www.fumadocs.dev/docs/mdx/next>

- 3 13 26 89 **Root Provider | Fumadocs**
<https://www.fumadocs.dev/docs/ui/layouts/root-provider>
- 4 69 94 **Introduction**
https://www.fumadocs.dev/docs/headless?utm_source=chatgpt.com
- 6 29 33 34 48 50 **Getting Started | Fumadocs**
<https://www.fumadocs.dev/docs/mdx>
- 8 47 73 74 86 **Server Entry | Fumadocs**
<https://www.fumadocs.dev/docs/mdx/entry/server>
- 9 43 44 45 46 77 83 93 **Page Slugs & Page Tree | Fumadocs**
<https://www.fumadocs.dev/docs/page-conventions>
- 10 31 32 49 71 **Loader API | Fumadocs**
<https://www.fumadocs.dev/docs/headless/source-api>
- 11 51 52 57 58 59 **Docs Layout**
https://www.fumadocs.dev/docs/ui/layouts/docs?utm_source=chatgpt.com
- 12 60 61 62 63 64 65 66 67 **Docs Page | Fumadocs**
<https://www.fumadocs.dev/docs/ui/layouts/page>
- 14 70 78 79 80 88 **Customize UI | Fumadocs**
<https://www.fumadocs.dev/docs/guides/customize-ui>
- 15 **Themes | Fumadocs**
<https://www.fumadocs.dev/docs/ui/theme>
- 16 **Components | Fumadocs**
<https://www.fumadocs.dev/docs/ui/components>
- 17 **Layout Links**
https://www.fumadocs.dev/docs/ui/layouts/links?utm_source=chatgpt.com
- 18 **Navbar | Fumadocs**
<https://www.fumadocs.dev/docs/ui/layouts/nav>
- 19 **Search | Fumadocs**
<https://www.fumadocs.dev/docs/search>
- 20 **Built-in Search | Fumadocs**
<https://www.fumadocs.dev/docs/headless/search/orama>
- 21 **TOC | Fumadocs**
<https://www.fumadocs.dev/docs/headless/components/toc>
- 22 **Breadcrumb | Fumadocs**
<https://www.fumadocs.dev/docs/headless/components/breadcrumb>
- 23 68 **Page Tree | Fumadocs**
<https://www.fumadocs.dev/docs/headless/page-tree>
- 25 **Getting Started: Server and Client Components**
https://nextjs.org/docs/app/getting-started/server-and-client-components?utm_source=chatgpt.com
- 27 53 54 55 56 72 81 85 **Docs Layout | Fumadocs**
<https://www.fumadocs.dev/docs/ui/layouts/docs>
- 35 87 **Functions: generateMetadata | Next.js**
<https://nextjs.org/docs/app/api-reference/functions/generate-metadata>

36 **Functions: notFound | Next.js**

<https://nextjs.org/docs/app/api-reference/functions/not-found>

37 90 **Browser Entry | Fumadocs**

<https://www.fumadocs.dev/docs/mdx/entry/browser>

38 39 40 41 42 **Collections | Fumadocs**

<https://www.fumadocs.dev/docs/mdx/collections>

75 **File-system conventions: not-found.js | Next.js**

<https://nextjs.org/docs/app/api-reference/file-conventions/not-found>

76 **Search UI | Fumadocs**

<https://www.fumadocs.dev/docs/ui/search>

91 **Layouts**

https://www.fumadocs.dev/docs/ui/layouts?utm_source=chatgpt.com

92 **llms.txt**

https://www.fumadocs.dev/llms.txt?utm_source=chatgpt.com